CS4800: Designing Sustainable ICT Systems

Lecture 4: Sustainable AI

September 25, 2025

Lecturer: Przemysław Pawełczak
Authors: Cristian Cuţitei, Liwia Padowska, Alex Despan, Przemysław Pawełczak

CS4800-EWI@tudelft.nl

TU Delft

What will you learn in this lecture?

Week 1.6: Sustainable AI

- Environmental and computational costs of AI: carbon, energy, FLOPs, parameters, runtime
- Role of accelerators beyond CPUs: GPUs, TPUs, ASICs, FPGAs, neuromorphic chips, TinyML
- Efficient training and learning methods: meta-learning, transfer learning, dataset standardization
- Efficient inference techniques: model distillation, quantization

Red vs Green AI: Why Sustainability Matters

Bigger isn’t always better. Sometimes, greener is smarter.

Red vs Green AI: philosophies

Red AI

- **Pursues** maximum accuracy by scaling data, parameters, and training time
- **Ignores** economic and environmental costs (energy, carbon, accessibility)
- **Delivers** diminishing returns where small gains demand exponentially more compute

Green AI

- **Pursues** competitive results with **minimum compute**
- **Accounts for** economic and environmental costs (energy, carbon, accessibility)
- **Delivers** responsible progress by optimizing efficiency and resource use

Red vs Green AI: measuring sustainability

Metrics:

- **Carbon emissions during training** — depend on region and energy source
- **Energy consumption** — recorded via GPUs or power meters
- **FLOPs** — hardware-independent measure of the total number of arithmetic operations required to train or run a model
- **Trainable parameters** — larger parameter counts usually require more memory, storage and longer training
- **Runtime** — total wall-clock time, including preprocessing and checkpointing

No single measure gives the full picture!

Red vs Green AI: toward transparent footprints

Problem: Today, AI's environmental footprints are **underreported**. Energy, carbon, and hardware details are rarely disclosed.

Make them transparent:

- Use **lightweight loggers** for real-time energy tracking
- Publish **environmental metrics** alongside accuracy
- Provide **experiment-level reporting** - release code or models (when appropriate) to avoid wasteful replication
- Train in **efficient environments** (e.g., keep RL loops on a GPU)

Rethink leaderboards: reward not just **accuracy**, but also **efficiency and carbon footprint**

Replication: re-running experiments to verify results. Without shared code, researchers must rebuild from scratch.

What hardware should sustainable AI be trained on?

AI Accelerator Hardware

Preamble: Three Walls that CPUs hit

Wall 1: ILP Saturation

- **Instruction-Level Parallelism (ILP)**: overlapping instructions via pipelining, out-of-order execution
- Gains slowed:
  - Dependencies between instructions (Bernstein's conditions)
  - More hardware complexity → diminishing returns
- Example: not every instruction stream has enough independent work to exploit

CPUs reached their limit in terms of speed gained by leveraging ILP

Preamble: Three Walls that CPUs hit

Wall 2: transistor count wall

- Dennard scaling broke down:
  - Shrinking transistors no longer reduced voltage proportionally
  - **Leakage currents** and **heat dissipation** grew uncontrollable
- Raising clock speeds → exponential power consumption
- Result: CPU clock frequency plateaued ~3–5 GHz

No more **“Free”** performance from higher clock speeds

Preamble: Three Walls that CPUs hit

Wall 3: Memory Wall

- CPUs rely on caches to hide slow main memory
- But **deep neural networks** and big-data workloads require:
  - Huge arrays of weights and activations
  - Regular, streaming access patterns
- Cache tricks are less useful → cores **need more bandwidth**

Memory, another huge bottleneck for throughput

Throughput: the number of data units processed in one unit of time

Overcoming CPU Walls

Why Are Accelerators Needed?

1. **ILP saturation** → we need masive **data/thread parallelism** (SIMD/SIMT)
2. **Power/frequency wall** → we need **specialized compute** and **near-data** execution to cut joules
3. **Memory wall** → we need **high-bandwidth memory** **and on-chip** **systolic dataflows** to maximize reuse and minimize transfers

Accelerators (GPUs, TPUs, etc.) exploit **parallelism and locality** → higher throughput and performance-per-watt for ML

Fabrication View

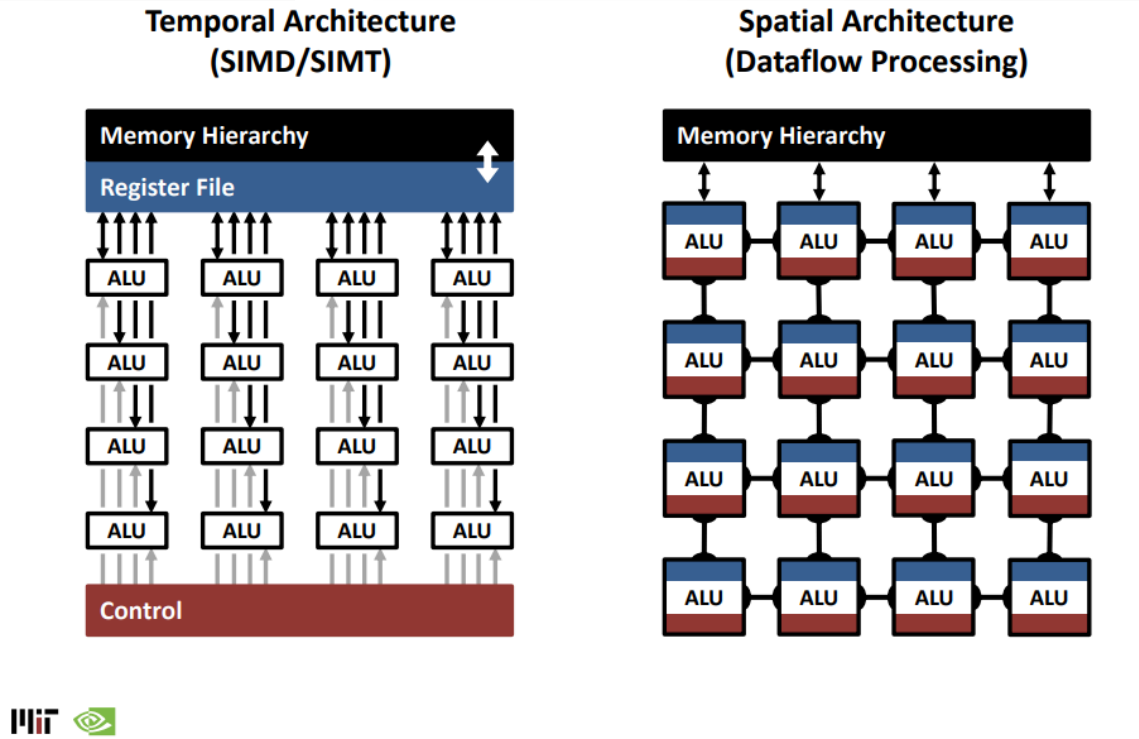
ASICs vs FPGAs

- **Application-Specific Integrated Circuits (ASICs)**
  Every wire is laid out for a **single purpose**, with logic fixed at fabrication
  - **Pros:** Very high efficiency; excellent performance-per-watt
  - **Cons:** Long design cycles; **inflexible** post-fabrication
- **Field-Programmable Gate Arrays (FPGAs)**
  Grids of **programmable logic blocks and interconnects**; can be rewired to new datapaths
  - **Pros:** Rapid prototyping; reprogrammable in software; **larger memory bandwidth than GPUs**,lower latency
  - **Cons:** Complex toolchains (VHDL/Verilog); costly at high volumes; **lower performance-per-watt than ASICs** (limited power optimization control); few open-source libraries (e.g., LeFlow)

Architectural paradigms

Temporal versus Spatial Architectures

- **Temporal** (CPUs, GPUs) — each ALU sequentially fetches data from shared memory and maps NN operations to matrix multiplications
- **Spatial** (e.g., TPUs, neuromorphic) — each ALU has its own local memory and control logic, allowing them to work in parallel



Temporal vs Spatial Architecture

Accelerator types: GPUs

Graphics processing units

Specialized processors for computer graphics; later adopted for highly parallel general-purpose computing such as AI

Integrated graphics processor — on the same chip as the CPU; uses system RAM
Dedicated graphics processor — separate chip (often on an expansion card) with its own high-speed video memory, cooling, and power delivery for very high parallel throughput

- **Pros**
  - Very high computational throughput for floating-point matrix calculations
  - Mature software support (common deep-learning libraries), versatile across workloads
- **Cons**
  - High electricity consumption, especially in large data centers
  - Often less energy-efficient than ASICs or reconfigurable chips for narrow tasks

AMD Ryzen 5 5600G (integrated) and IMSI GeForce RTX 4060 (dedicated)

Accelerator types: ASICs

Application-specific integrated circuits

Purpose-built chips: fixed “assembly line” of math units, no reconfiguration is possible

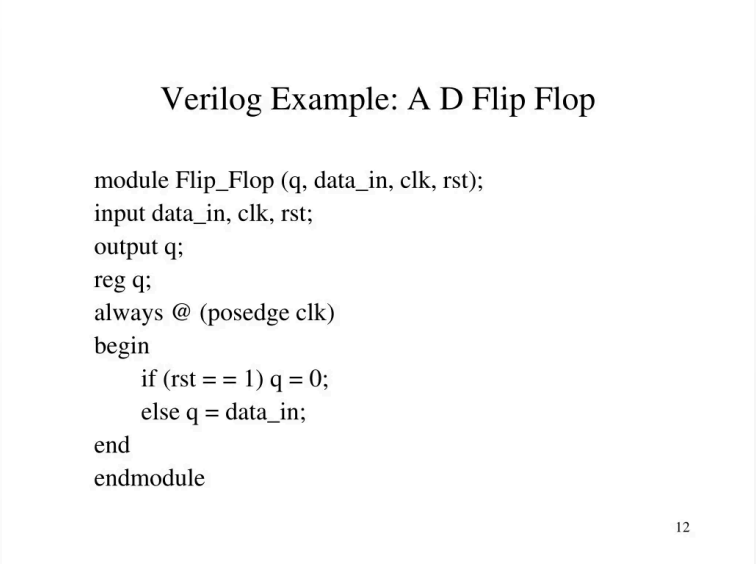
- **Pros**
  - Highest efficiency for deep neural networks
  - Best throughput and lowest latency
  - Low runtime power draw
- **Cons**
  - Very costly to design and fabricate
  - Long development time before release
  - Fixed hardware, cannot adapt to new models

Accelerator types: FPGAs

Field-programmable gate arrays

Reconfigurable hardware: like digital LEGO blocks, load a file to define circuits

- **Pros**
  - Adaptable to new model types
  - Lower energy use than graphics processors
  - Cheaper than ASICs for small/medium production runs
  - Balance of flexibility and efficiency
- **Cons**
  - Lower peak performance than ASICs
  - Toolchains more complex, slower to develop
  - Limited memory and compute for very large models



Verilog example

Accelerator types: Neuromorphic chips

Neuromorphic Computing

Brain-inspired chips: neurons fire spikes only when needed, event-driven

- **Pros**
  - Extremely low power use
  - Efficient for sparse, real-time sensor data
  - Parallel, low-latency, adaptive learning possible
  - Ideal for edge devices with limited power/bandwidth
- **Cons**
  - Immature software and tools
  - Not compatible with most mainstream DL models
  - Few chips available, small research community

NeuroShield (NM500) chip with 567 neurons

Accelerator Types — TPUs

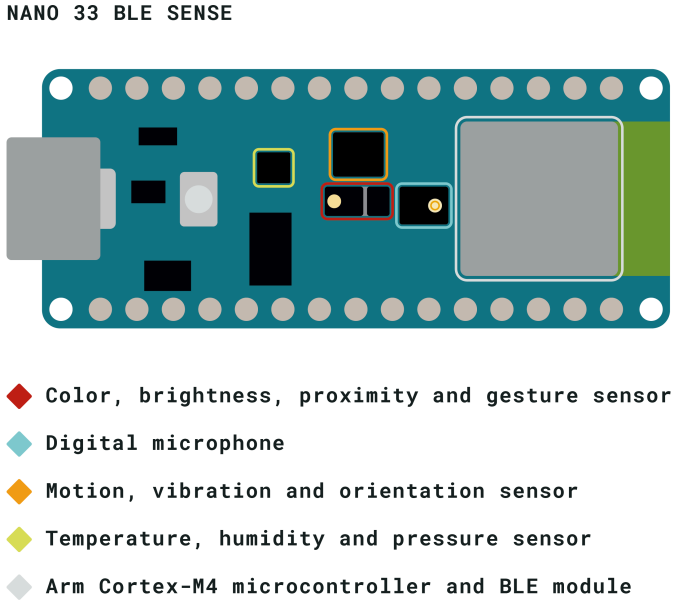
Tensor Processing Units

- **Core:** systolic array
  - Inputs stream row-wise, weights column-wise
  - Partial results move diagonally, enabling reuse
- **Memory hierarchy**
  - On-chip buffer, fast memory beside the array that holds activations, lowering latency
  - Accumulators in each processing element
- **Strengths**
  - Optimized for matrix multiplications and convolutions
  - Compiled via **XLA** for predictable scheduling
  - Custom instructions: device-specific ops (e.g. reductions) tailored to tensors

Ultra-low-power edge AI

TinyML

- ML on microcontrollers or sensors (from kB to MB RAM)
- Tooling: TensorFlow Lite Micro, CMSIS-NN, etc.



Nano 33 BLE Sense

Key Takeaways

- Report **compute, energy, and carbon**, not only accuracy
- CPUs face **ILP, power and memory walls**
- Accelerators improve efficiency via **parallelism and data locality**
- **Fabrication** (ASIC vs. FPGA) is distinct from **architecture** (temporal vs. spatial)
- **GPUs** provide broad throughput; **ASICs** maximize performance per watt; **FPGAs** offer reconfigurability
- **Neuromorphic** computing targets ultra-low power; **TinyML** enables ML on microcontrollers

Metalearning

What is it? And how exactly is it sustainable?

Classically defined as learning to learn

- Hyperparameter Tuning
- Neural Architecture Search (NAS)

Leverages the experience of conducting multiple ML experiments to speed up the training process

The problem that any AI engineer faces

Hyperparameter tuning

It affects how well the model trains, therefore it has an effect on final performance. Smaller model with tuned hyperparameters outperforms a larger model with poor tuning.

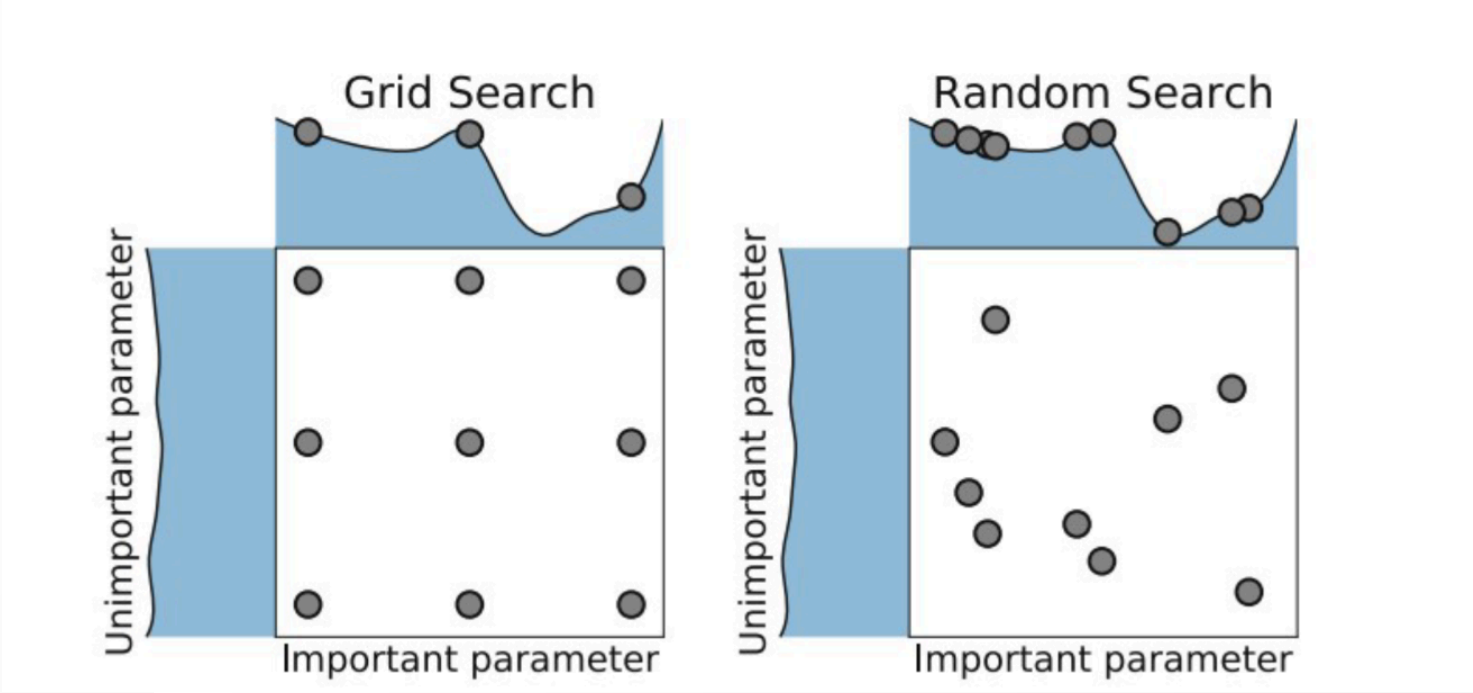
Take classic Support Vector Machine (SVM)

- Tune 15 hyperparameters
- Use Grid Search with 4 values/hyperparameter
- Leads to **1,073,741,824 unique combinations** ($N_{\text{runs}} = \prod_{i=1}^{k} |H_i|$, k is params, H vals)
- **IF** it takes only one second to train — approx **34 Years** to check all combinations

Not a good solution, we need a better way to look for hyperparameters, enter **Random Search**

Random search

- Uniformly sample between two boundaries
- Restart anytime, just need to keep in mind previous best
- Does not care how many hyperparameters you tune



A smarter way to do it

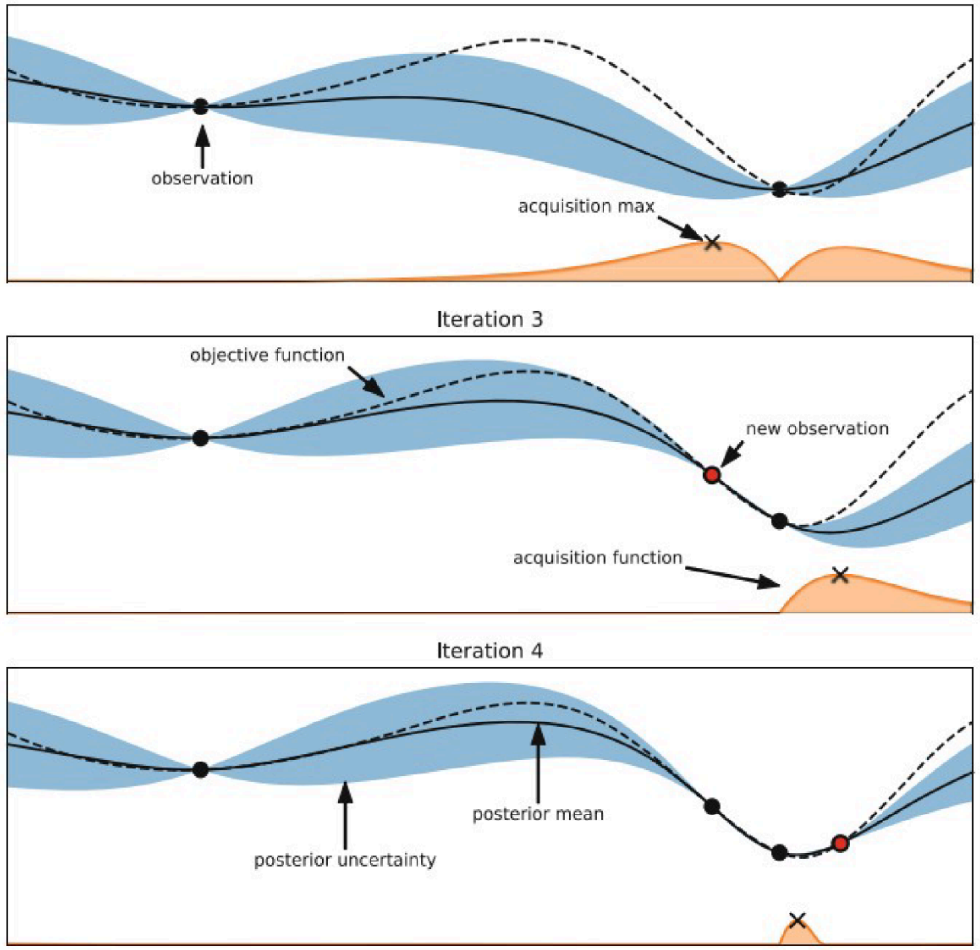
Model the performance curve

- Sample hyperparameters
- Model performance
- Test promising points and update model

Bayesian Regression

- Non-linear
- Confidence intervals

Dark line: mean prediction
Blue band: confidence interval
Black dots: sampled hyperparameter values
Curve fit: suggests promising regions

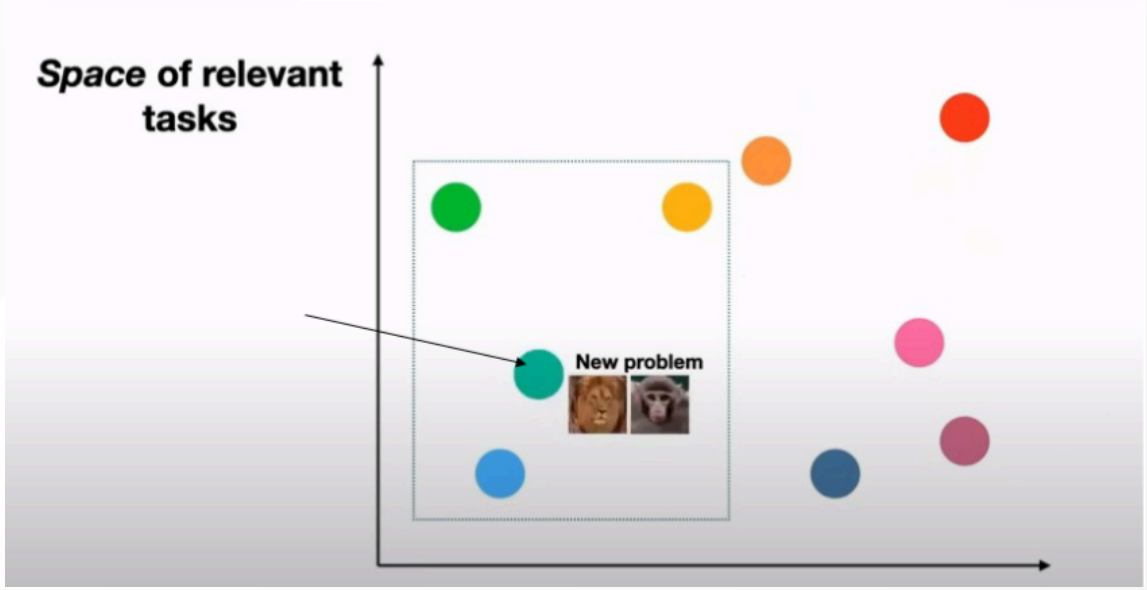


TU Delft Course

Portfolio Design

Idea: Create feature space for datasets to live in

Just like feature vectors for images or text. This space is used to store features about datasets, and the lables are their successful hyperparameters



TU Delft Course

These features can be anything extracted from the dataset:

- Hand-crafted features: NumberOfInstances, Minority/MajorityClassSize etc.
- Landmarkers: performance of a simple classifier like Decision Trees etc.

Techniques for reducing carbon footprint of AI

Sustainable datasets

Problem: dataset reproducibility problem

The process of dataset creation is resource intensive: no plan, leads to inefficiency

Examples of sheer scale of datasets:

- ImageNet: large-scale visual database containing **over 14 million images**
- Waymo Open: a large-scale autonomous driving dataset. Thousands of hours of high-resolution video, LiDAR scans, GPS data, and sensor readings

Problem: data redundancy

You always hear the issue that "there is not enough data", when in fact, quality is more often than not, better than quantity

Sustainable datasets

Solution: dataset reproducibility problem

Standardization of the process for creating and sharing datasets. Akin to the ideas proposes in the paper "Datasheets for datasets"

Solution: redundant training

The Embedding Range Judgement method

Fancy words but the idea is simple:

- Embed the whole dataset in feature space
- Take only the samples that are far apart

Sustainable training

Problem: What if you can not acquire enough data?

Some data can be so rare that it is impossible to get more of. This issue can stem from lack of resources or from the fact that the event observed is simply infrequent.

Take for example:

- high-resolution medical imaging data for rare brain diseases
- defective products compared to millions of defect-free examples
- annotated images for endangered species

Sustainable training

Solution: Transfer learning

"If you can ride a bike, it's easier to learn to drive a motorcycle"

Data for one task or domain is reused as data on a different, but related, domain. All you have to do is shift the domain to fit the distribution of the small data you have.

Fine tuning

The most popular transfer learning technique

- splitting into pre-training + fine-tuning amortizes large upfront costs; fine-tuning is cheap
- companies prefer fine-tuning multiple models over building larger ones (mixture-of-experts)
- works well in vision: e.g., swap out last ImageNet layers, retrain in hours (instead of days)

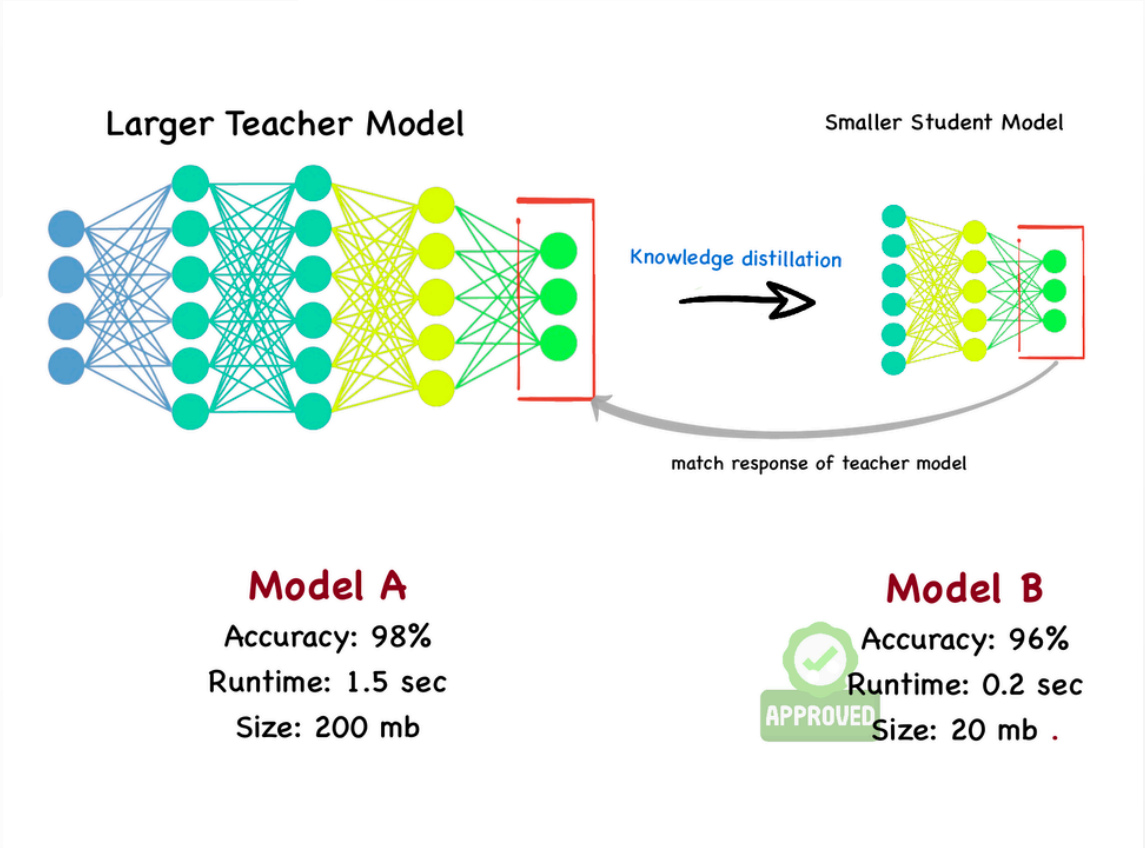
Sustainable inference

Problem: inference is where most emissions are created

Solution: model distillation

Train a smaller model to have similar performance to older, bigger model.

How? Instead of using the training labels as prediction targets, you use the class probabilities



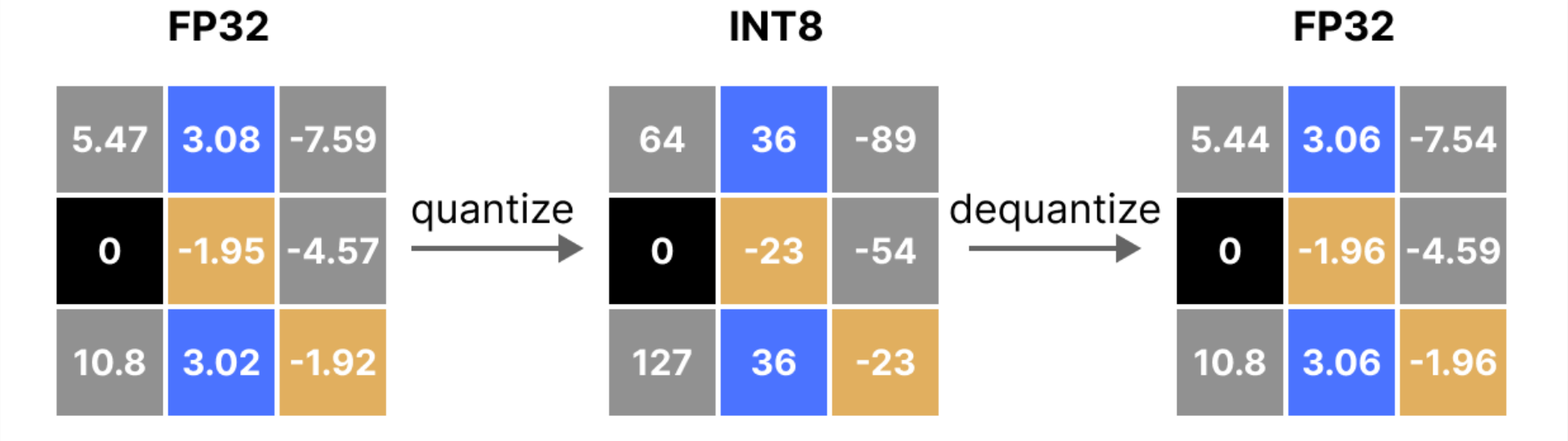
Model distillation

Sustainable inference

Solution: quantization

You can make the model you already have smaller with little to no extra training.

Remember weights? 32-bit Floats, do we need the whole 32 bits?
Spacial costs reduce linearly, but computational costs quadratically (matrix multiplications).



Qunatize to from 32-bit float to 8-bit int

Sustainable inference

Types of quantization

Post Training Quantization (PTQ) and **Quantization Aware Training (QAT)**.

Post training quantization

Requires little/no additional training, and is usually an one line solution in many libraries
This usually leads to a satisfactory result using 8-bit weights

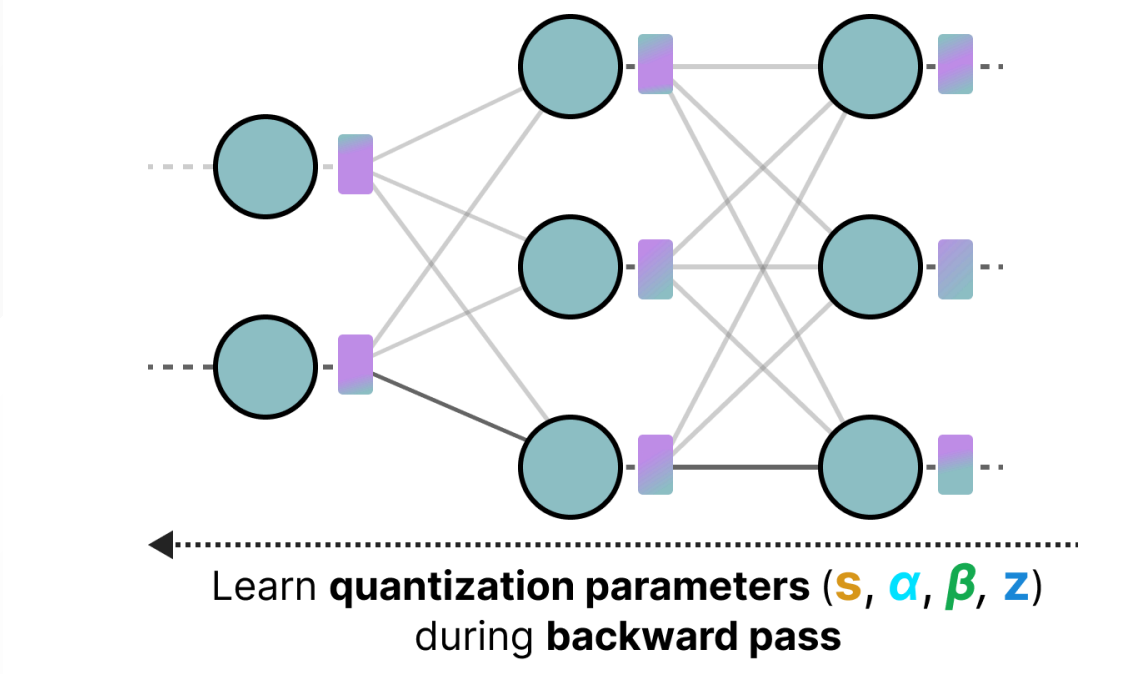
- adjust quantizing parameters to optimize for values being as close as possible to original, and preserve classification order
- Additional steps you might want do is to correct for bias that appears when quantizing (clipping and rounding error)

Sustainable inference

Quantization aware training

This technique has been show to be able to achieve 4-bit weights, an 8 fold memory and 16 fold computational decrease

As an additional plus to speed up training, one can and should employ the techniques used in **PTQ** as an initialization step for this algorithm



s: scale factore, α: bottom bound, β: top bound, z: zero point

Recap

- Sustainability = report **compute, energy, carbon** (not just accuracy)
- CPUs hit ILP/power/memory walls → **accelerators** recover scaling
- **Fabrication ≠ architecture**; choose GPU/ASIC/FPGA per need
- **Meta-learning** reduces tuning waste; **portfolio** reuses prior wins
- **Transfer learning** cuts data/compute; **distillation & quantization** shrink inference costs

Feedback

- This is the first edition of the course - we need your feedback!
- Feedback on Lecture 4 slides/notes:

Example Exam Questions

1) Which is *not* a typical reason GPUs help ML?

A. High data parallelism
B. Mature DL toolchains
C. Always lowest energy use
D. High throughput

2) Which statement about ASICs is TRUE?

A. They can be reconfigured after fabrication
B. They deliver high perf/W for specific tasks
C. They’re universally better than GPUs for all tasks
D. They’re cheaper to iterate than FPGAs

Example Exam Questions

1) Which is *not* a typical reason GPUs help ML?

A. ~~High data parallelism~~

B. ~~Mature DL toolchains~~

C. **Always lowest energy use**

D. ~~High throughput~~

Answer: C

2) Which statement about ASICs is TRUE?

A. They can be reconfigured after fabrication

B. They deliver high perf/W for specific tasks

C. They’re universally better than GPUs for all tasks

D. They’re cheaper to iterate than FPGAs

Example Exam Questions

1) Which is *not* a typical reason GPUs help ML?

A. ~~High data parallelism~~

B. ~~Mature DL toolchains~~

C. **Always lowest energy use**

D. ~~High throughput~~

Answer: C

2) Which statement about ASICs is TRUE?

A. ~~They can be reconfigured after fabrication~~

B. They deliver high perf/W for specific tasks

C. ~~They’re universally better than GPUs for all tasks~~

D. ~~They’re cheaper to iterate than FPGAs~~

Answer: B

Thank you for your attention!

Next lecture: Sustainable Data Centers

Thursday, October 09, 15:45-17:30 EEMCS Hall F

Lecture 4 summary

- Data center architecture: computing, cooling, power
- Environmental footprint of cloud computing, measurement methods
- Industry trends: compute growth, efficiency gains, energy
- Prospective solutions: cloud strategy, demand response, cooling technologies, waste heat reuse

Announcement

Next week, Thursday, October 02 at 15:45-17:30, MANDATORY midterm presentations

Midterm presentation (mandatory, ungraded): formative assessment, expect feedback

- Thursday, October 2, 15:45, EEMCS Hall F
- Slides: introduction, chosen topic, group progress

Good luck!